EXP 9

```
clc;
clear;
close all;

% Performance Evaluation of AM Generation and Detection
% under Noisy Channel Conditions Using MATLAB

%% 1. System Parameters
Fs = 200000;              % Sampling frequency (Hz)
T = 0.05;                 % Simulation duration (seconds)
t = 0:1/Fs:T-1/Fs;

fm = 200;                 % Message frequency (Hz)
fc = 10000;               % Carrier frequency (Hz)
Am = 1;                   % Message amplitude
Ac = 1;                   % Carrier amplitude
mu = 0.7;                 % AM modulation index

SNR_dB = [0 5 10 15 20];     % SNR values in dB

%% 2. Generate Message Signal
m = Am*cos(2*pi*fm*t);

%% 3. Generate AM Signal
sAM = Ac*(1 + mu*m/Am).*cos(2*pi*fc*t);

%% 4. Plot Original Signals
figure('Name','AM Signal Generation');

subplot(2,1,1);
```

```
plot(t,m,'b','LineWidth',1.2);
title('Original Message Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(2,1,2);
plot(t,sAM,'r');
title('AM Modulated Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

%% 5. Initialize Performance Variables
MSE = zeros(size(SNR_dB));
OutputSNR = zeros(size(SNR_dB));

% Low-pass filter for envelope smoothing
[b,a] = butter(5, 1500/(Fs/2), 'low');

%% 6. Noisy Channel and AM Detection
figure('Name','AM Detection Under Noise');

for k = 1:length(SNR_dB)

    % Add AWGN
    signalPower = mean(sAM.^2);
    noisePower = signalPower / (10^(SNR_dB(k)/10));
    noise = sqrt(noisePower)*randn(size(sAM));
    received = sAM + noise;
```

```
% Envelope detection using Hilbert transform
envelope = abs(hilbert(received));

% Remove DC component
detected = envelope - mean(envelope);

% Low-pass filtering
detected = filtfilt(b,a,detected);

% Normalize to message amplitude
detected = detected / mu;

% Remove small numerical offset
detected = detected - mean(detected);

% Calculate MSE
errorSignal = m - detected;
MSE(k) = mean(errorSignal.^2);

% Estimate output SNR
OutputSNR(k) = 10*log10( ...
   mean(m.^2)/(mean(errorSignal.^2) + eps));

% Plot received and detected signals
subplot(length(SNR_dB),1,k);
plot(t,m,'b','LineWidth',1.1);
hold on;
plot(t,detected,'r');
title(sprintf('AM Detection at Input SNR = %d dB', ...
   SNR_dB(k)));
xlabel('Time (s)');
```

```
    ylabel('Amplitude');
    legend('Original','Detected');
    grid on;

end

%% 7. Performance Evaluation
figure('Name','AM System Performance');

subplot(2,1,1);
plot(SNR_dB,MSE,'bo-','LineWidth',1.5);
title('Mean Squared Error vs Input SNR');
xlabel('Input SNR (dB)');
ylabel('MSE');
grid on;

subplot(2,1,2);
plot(SNR_dB,OutputSNR,'rs-','LineWidth',1.5);
title('Output SNR vs Input SNR');
xlabel('Input SNR (dB)');
ylabel('Output SNR (dB)');
grid on;

%% 8. Display Results
fprintf('\nAM Performance Evaluation Results\n');
fprintf('-----------------------------------------------\n');
fprintf('Input SNR (dB)    MSE       Output SNR (dB)\n');
fprintf('-----------------------------------------------\n');

for k = 1:length(SNR_dB)
    fprintf('%10.1f    %8.5f    %10.2f\n', ...
```

```
    SNR_dB(k), MSE(k), OutputSNR(k));
end

disp('AM generation and noisy-channel evaluation completed.');
```

